CNN & Transfer Learning

Animal Classification

Using Animals-10 Dataset consisting 10 Classes, ~26,000 Image

Table of Contents

1. Data Set
2. Preprocessing Pipeline
3. Data Augmentation
4. Model Architectures
5. Training Process
6. Model Export Formats
7. Flask Application
8. Docker Deployment

Introduction

This report documents the end-to-end process of training four deep learning models for image classification on the **Animals-10** dataset, and deploying them together inside a local web application built with **Flask** and containerised with **Docker Compose**. The workflow spans data ingestion and normalisation, exploratory analysis, model design and training across two Jupyter notebooks (one for the vanilla CNN and one for the three transfer-learning models), model export, and finally serving all four models through a single web interface where the user can upload an image and choose which network to run inference with.This report details the comprehensive process of training four deep learning models for image classification utilizing the **Animals-10** dataset. Furthermore, it covers the deployment of these models within a localized web application constructed with **Flask** and containerized using **Docker Compose**. The workflow encompasses data ingestion and normalization, exploratory data analysis, model design and training across two distinct Jupyter notebooks (one dedicated to the vanilla CNN and the other to the three transfer-learning models), model export, and, ultimately, the serving of all four models via a unified web interface. This interface permits users to upload an image and select the desired network for inference execution.

1. Dataset – Animals-10

Animals-10 is a publicly available dataset hosted on HuggingFace (34data/animal-10) containing approximately 26 000 images across ten animal categories. Original folder names are in Italian; they are mapped to English at training and inference time.

cane	dog	mucca	cow
cavallo	horse	pecora	sheep
elefante	elephant	ragno	spider
farfalla	butterfly	scoiattolo	squirrel
gallina	chicken	gatto	cat

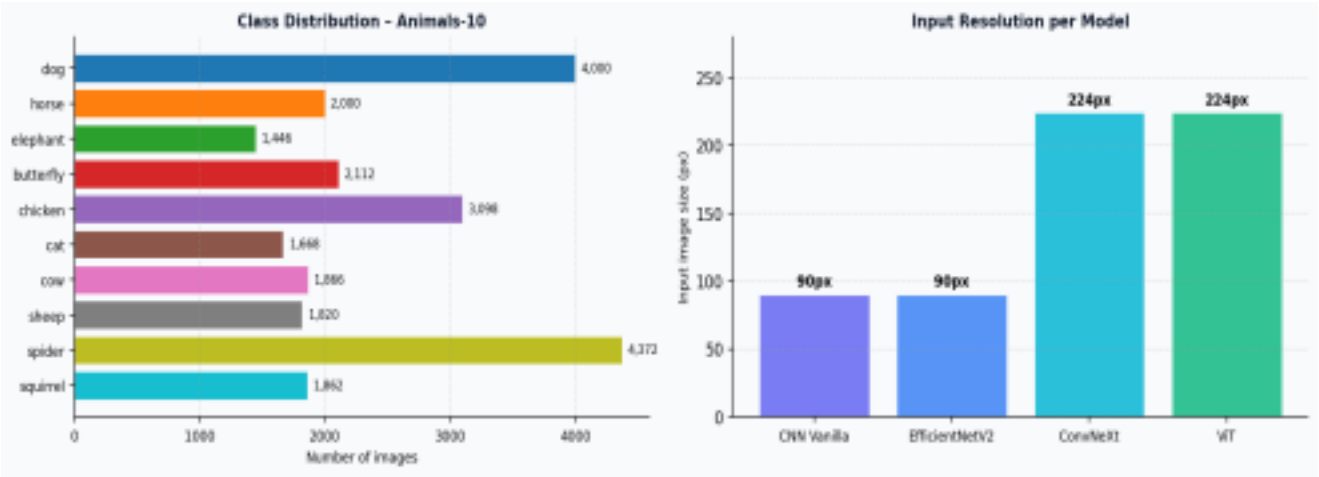

Fig. 1 – Left: approximate image count per class. Right: input resolution required by each model.

2. Preprocessing Pipeline

All images are downloaded via the HuggingFace datasets library, converted to RGB, resized to a common target size, and saved as JPEG files organised into per-class subfolders. This normalised folder structure is then consumed directly by

tf.keras.utils.image_dataset_from_directory()

or

torchvision.datasets.ImageFolder.

Fig. 2 – Full preprocessing and training pipeline from raw dataset to exported model.

Key preprocessing parameters

- **Target size:** 90 × 90 px (stored in TARGET_SIZE variable)
- **Colour mode:** RGB (forced conversion via PIL)
- **JPEG quality:** 92 when saving to disk
- **Train / val split:** 80 % / 20 % with seed = 42
- **Note:** the output folder is named normalized-224 but images are 90 × 90 px — ConvNeXt and ViT up-scale to 224 at load time.

3. Data Augmentation

Augmentation is applied **only to the training split**, inline inside the tf.data pipeline (or as PyTorch transforms for ViT). The validation set always receives only the model-specific preprocessing function — no random transforms.

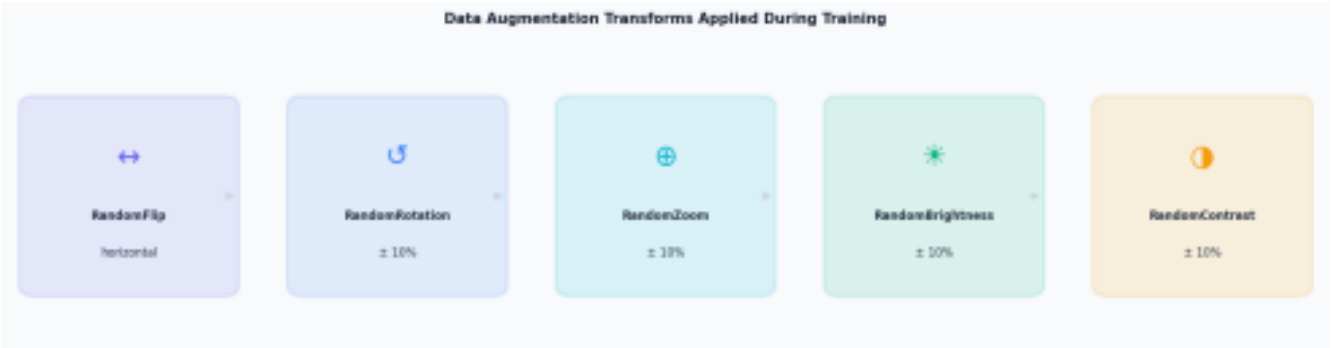

Fig. 3 – Data augmentation transforms applied during training. ConvNeXt and ViT use all five; CNN Vanilla uses Flip and Rotation only.

Per-model augmentation details

Model	RandomFlip	Random Rotation	Random Zoom	Brightness / Contrast
CNN Vanilla	horizontal	10 %	—	—
EfficientNetV2	horizontal	—	—	—
ConvNeXt	horizontal	10 %	10 %	Contrast 10 %
ViT-B/16	horizontal	—	80 – 100 % crop	Brightness + Contrast

Preprocessing function per model

Each model family expects a different pixel value range. Applying the wrong normalisation at inference time leads to completely incorrect predictions, so it is critical that training and inference use the same function.

Model	Training Preprocessing	Value Range Output
CNN Vanilla	Rescaling (1/255)	[0.0 – 1.0]
EfficientNetV2	efficientnet_v2.preprocess_input()	ImageNet mean/std normalised
ConvNeXt	convnext.preprocess_input()	ImageNet mean/std normalised
ViT-B/16	ViTImageProcessor (HuggingFace)	ImageNet mean/std normalised

4. Model Architectures

Model	Framework	IMG Size	Preprocessing	Saved format
CNN Vanilla	TF / Keras	90×90	Rescaling /255	.keras
EfficientNetV2S	TF / Keras	90×90	efficientnet_v2 preprocess_input	.h5
ConvNeXtBase	TF / Keras	224×224	convnext preprocess_input	folder + .keras + label_map.json
ViT-B/16	PyTorch / HuggingFace	224×224	ViTImageProcessor (automatic)	HF folder config + safetensors

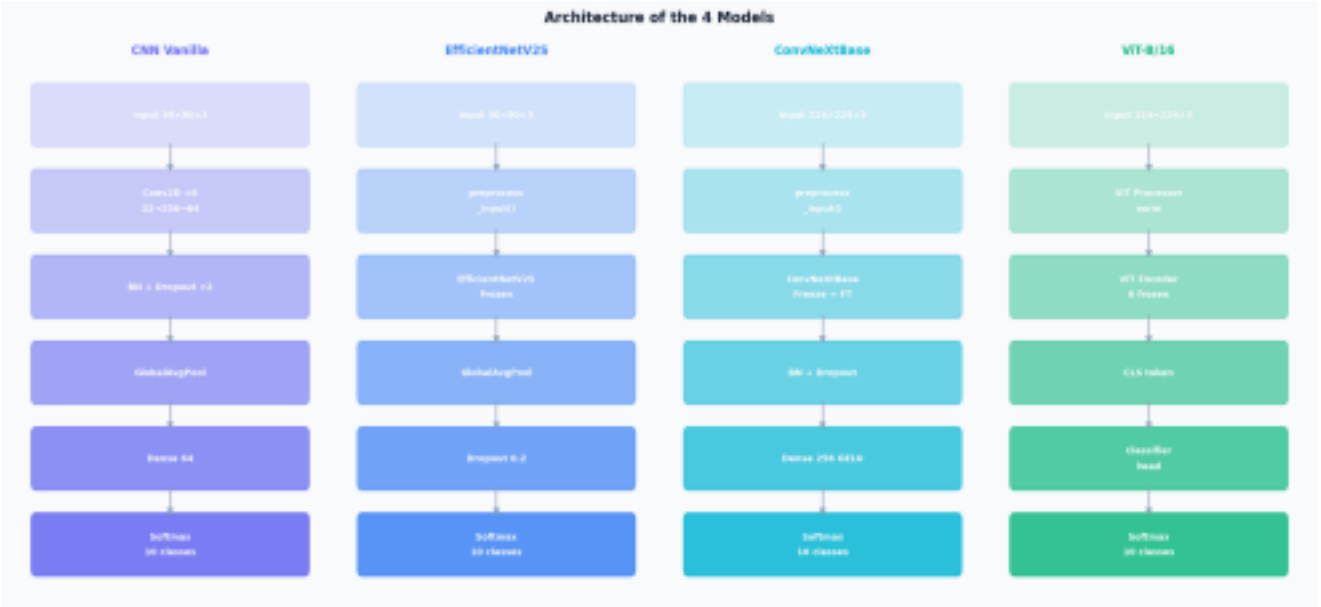

Fig. 4 – Layer-by-layer architecture diagram for each of the four models.

CNN Vanilla – custom architecture

A six-block convolutional network built from scratch with TensorFlow/Keras. Filters grow from 32 to 256 and then shrink back to 64, with Batch Normalisation and Dropout (0.3) applied at two checkpoints. Global Average Pooling replaces Flatten to reduce overfitting, followed by a 64-unit Dense head.

Transfer Learning models – shared strategy

All three transfer-learning models follow the same two-phase strategy: (1) **head training** — the pre-trained base is frozen and only the classification head is trained for a number of epochs; (2) **fine-tuning** — the last N layers of the base are unfrozen and trained at a much lower learning rate. This avoids destroying the ImageNet features learned by the base while allowing the model to specialise on the Animals-10 distribution.

5. Training Process

Fig. 5 – Training phase timeline for each model. ConvNeXt is the only one with an explicit two-phase schedule.

Model	Optimizer	LR	Loss	Epochs	Early Stop
CNN Vanilla	SGD (mom 0.9)	0.001	sparse_cat_CE	50	patience=5
EfficientNetV2	AdamW	1e-3	cat_CE	10	—
ConvNeXt	AdamW	5e-4 1e-5 (FT)	cat_CE	10+6	patience=3
ViT-B/16	AdamW	2e-5	CE + label smooth	10	patience=3

Callbacks used across all models

- **EarlyStopping** – monitors val_loss (Keras) or val_loss (PyTorch custom loop); restores best weights on stop. • **ReduceLROnPlateau** – halves the learning rate after 2 epochs without improvement (Keras models). • **ModelCheckpoint** – saves the best checkpoint to disk during training (ConvNeXt and ViT). • **TensorBoard** – logs loss and accuracy histograms for CNN Vanilla.
- **Linear LR Warmup** – 10 % of total steps used as warm-up for ViT (HuggingFace scheduler). • **Gradient clipping** – max norm 1.0 applied in ViT training loop.
- **Mixed precision** – float16 enabled for ConvNeXt to speed up GPU/TPU training.

ViT – custom PyTorch training loop

Unlike the three Keras models, ViT is trained with a hand-written epoch loop using torch.cuda.amp.autocast and GradScaler for automatic mixed precision. The first six encoder layers are frozen, and the classifier head uses a 5× higher learning rate than the rest of the encoder to accelerate convergence. Label smoothing (0.1) is applied to the cross-entropy loss.

6. Model Export Formats

Each model is saved in the format native to its framework. The Flask application must handle all three formats transparently.

CNN Vanilla

```
d_model.save('./vanilla_cnn.keras')
# Single .keras file containing architecture + weights
```

EfficientNetV2S

```
model.save('/content/drive/MyDrive/modelos/mi_modelo_animals10_efficientne
tv2s.h5') # Single .h5 (legacy HDF5 format)
```

ConvNeXtBase

Exports a folder containing model.keras plus a label_map.json file with the index-to-label mapping in the format {"index_to_label": {"0": "butterfly", ...}}. A training metrics plot is also included.

```
best_model.save(os.path.join(EXPORT_DIR, 'model.keras'))
with open(os.path.join(EXPORT_DIR, 'label_map.json'), 'w') as f:
    json.dump(label_info, f, indent=2)
```

ViT-B/16

Saved in HuggingFace format — a folder containing config.json (with the id2label mapping), model.safetensors, and preprocessor_config.json.

```
best_model.save_pretrained(FINAL_MODEL_DIR)
processor.save_pretrained(FINAL_MODEL_DIR)
```

Expected models/ folder structure

```
models/
├── vanilla_cnn.keras
├── animals10_efficientnetv2s.h5
├── convnext-model/
│   ├── model.keras
│   └── label_map.json
└── vit-model/
    ├── config.json
    ├── model.safetensors
    └── preprocessor_config.json
```

7. Flask Application

The web application is a single-file Flask app (main.py) that automatically discovers all available models, exposes a model-selection dropdown, and runs inference on uploaded images. The two key routes are:

Route	Method	Description	Template
/	GET	Home – model dropdown	index.html
/predict	POST	Receive image + model_id, run inference	result.html

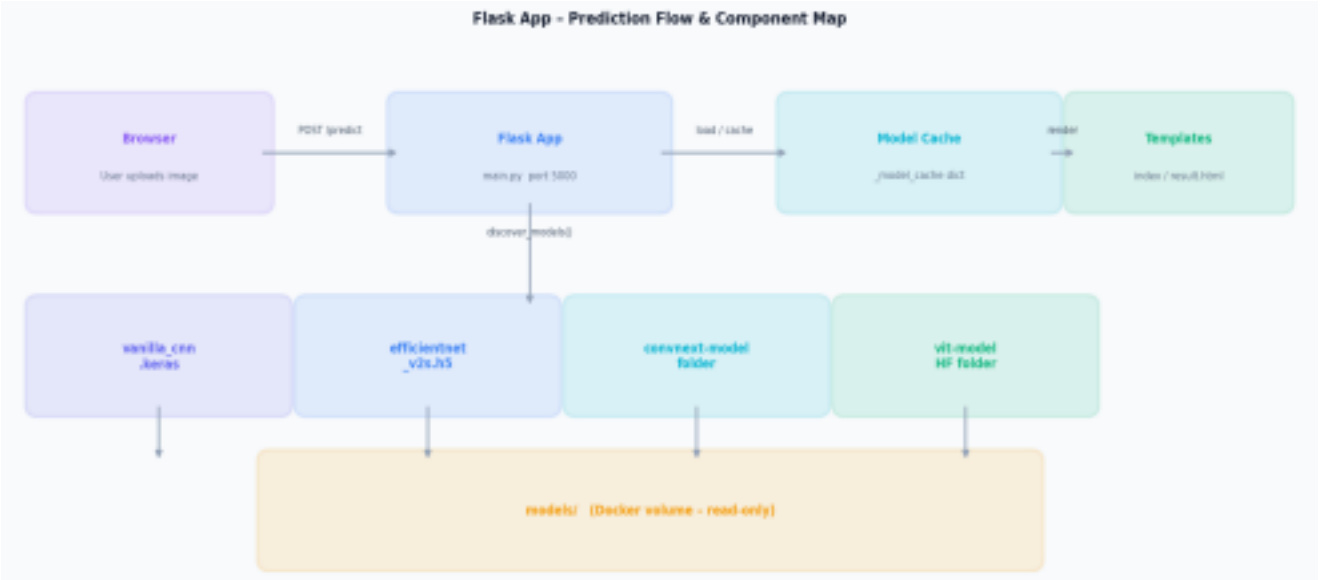

Fig. 6 – Component map and request flow of the Flask application.

Model discovery – discover_models()

At every request, discover_models() scans MODELS_ROOT and builds a list of available models by detecting three patterns:

- **HuggingFace folder** — any subfolder containing both config.json and model.safetensors → kind: transformers.
- **Keras .h5 file** — any *.h5 file → kind: keras.
- **Keras .keras file** — any *.keras file → kind: keras.

Each discovered model is assigned a slug ID, a human-readable name, the file/folder path, and the **preprocessing type** inferred from the filename (efficientnet, convnext, or rescale).

Model caching

Models are loaded lazily on first use and stored in a process-level dictionary _model_cache. Subsequent requests for the same model skip the disk read entirely, keeping inference latency low.

Preprocessing at inference time

The function _apply_keras_preprocess() selects the correct normalisation based on the preprocessing tag stored in the model metadata. This ensures that the pixel values fed to the network match exactly what the model saw during training:

```
def _apply_keras_preprocess(arr, preprocess_type):
    if preprocess_type == 'efficientnet':
        from tensorflow.keras.applications.efficientnet_v2 import preprocess_input
        return preprocess_input(arr)
    if preprocess_type == 'convnext':
        from tensorflow.keras.applications.convnext import preprocess_input
        return preprocess_input(arr)
    return arr / 255.0 # CNN Vanilla
```

Label resolution

Class names are resolved through a three-level fallback chain:

- **Level 1** – flat list in <model>.labels.json.
- **Level 2** – label_map.json with index_to_label dict (ConvNeXt export format).
- **Level 3** – hardcoded ANIMALS10_LABELS list when the model has exactly 10 outputs (covers CNN Vanilla and EfficientNetV2).

For ViT, labels are read from id2label inside config.json, which is populated automatically when save_pretrained() is called.

Prediction output

Both Keras and Transformer inference paths return a probability vector which is post-processed uniformly:

- The **top-1 prediction** (highest probability class) is shown prominently.
- The **top-3 predictions** with their confidence percentages are displayed as a ranked list.
- A **mood indicator** is shown: win ≥ 80 %, thinking ≥ 50 %, oops below 50 %.
- The uploaded image is shown alongside the results as a base64 preview.

Full prediction flow summary

Step	Function	Description
1	POST /predict	Receive image file + model_id from form
2	get_selected_model()	Look up model metadata from discovered list
3	build_image_preview()	Encode image as base64 for HTML preview
4	preprocess_image()	Open with PIL, convert to RGB
5	get_model_bundle()	Load model (or return from cache)
6	_apply_keras_preprocess() or HF processor	Apply correct normalisation
7	model.predict() / model(**inputs)	Run forward pass
8	_ensure_probability_vector()	Softmax if logits, clip to [0,1]
9	top-1 + top-3 extraction	Sort probabilities, map to class names
10	render result.html	Return HTML with predictions + image preview

8. Docker Deployment

The application is fully containerised with Docker Compose. The Dockerfile uses **python:3.11-slim** as base image — this is a deliberate choice because TensorFlow does not yet support Python 3.13/3.14, which may be the system default on the host. Docker isolates the runtime entirely, so no Python version conflicts occur.

Fig. 7 – Docker Compose stack layers, from host OS to browser.

Volume mapping

CNN Vanilla	./models/vanilla_cnn.keras	/app/model-store/vanilla_cnn.kera s	.keras file
EfficientNetV2	./models/animals10_efficientnetv 2s.h5	/app/model-store/animals10_effici entnetv2s.h5	.h5 file
ConvNeXt	./models/convnext-model	/app/model-store/convnext-model	folder
ViT-B/16	./models/vit-model	/app/model-store/vit-model	HF folder

Key configuration

- **MODELS_ROOT** env var → /app/model-store inside the container.
- **MODEL_ID** env var → sets the default selected model in the dropdown (e.g. hf-vit-model). • **All volumes are read-only** (:ro) — the app never writes to the model files.
- **restart: unless-stopped** — the container restarts automatically on reboot.
- **MAX_CONTENT_LENGTH** → 8 MB upload limit enforced by Flask.

Build and run

```
# 1. Make sure Docker Desktop is open and running

# 2. Build the image and start the container
docker compose up --build

# 3. Open the app in your browser
http://127.0.0.1:5000

# 4. Stop the container
docker compose down
```

The project is fully operational. All four models are served from a single Flask endpoint with correct preprocessing, label resolution, and model discovery — containerised and ready to run with a single command.